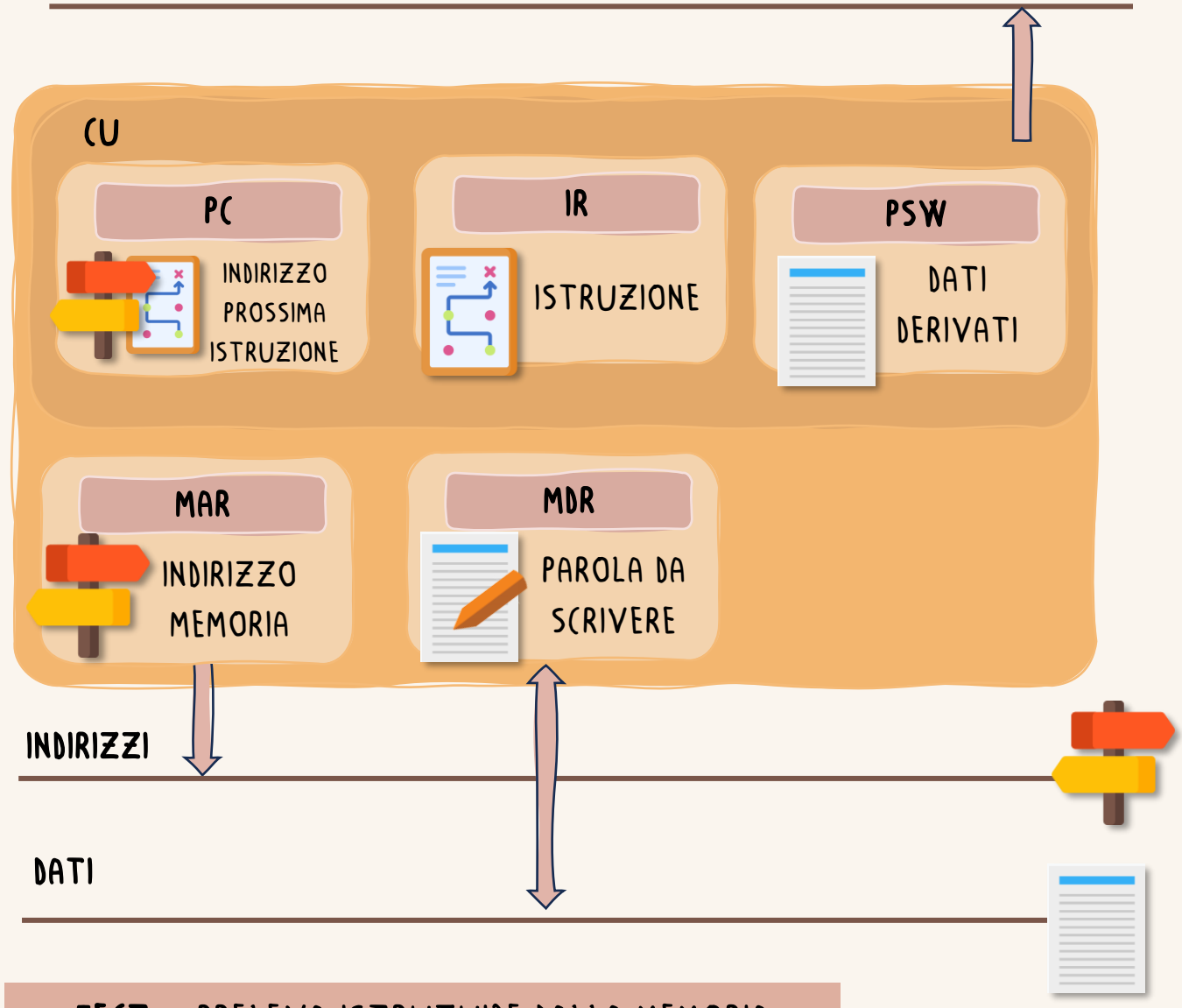
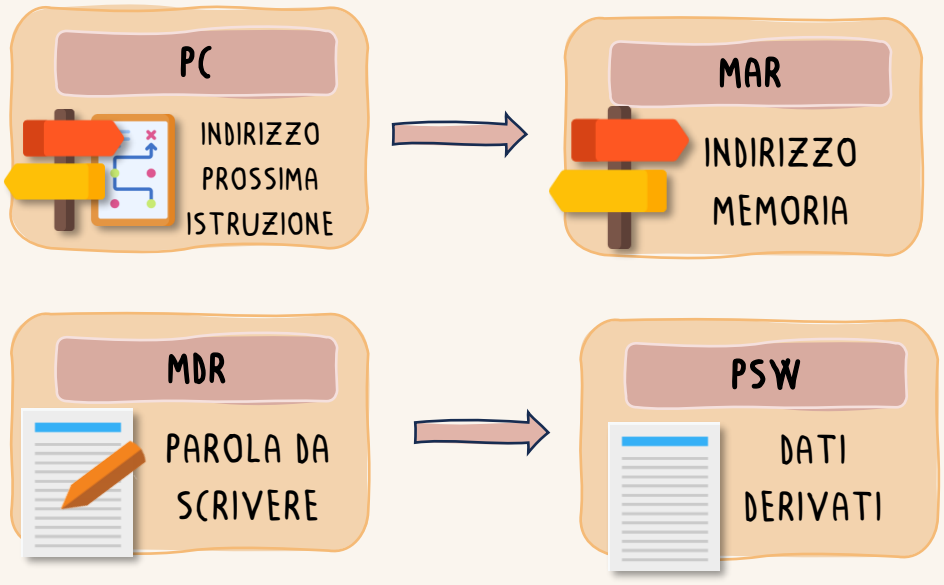
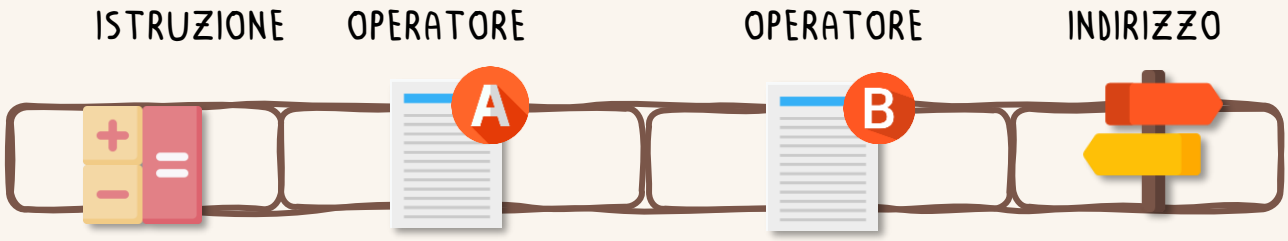
CPU



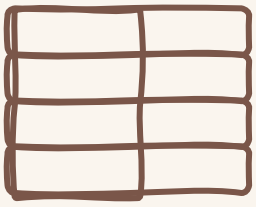
FECT – PRELEVA ISTRUZUIBE DALLA MEMORIA



ISTRUZIONE



REGISTRI



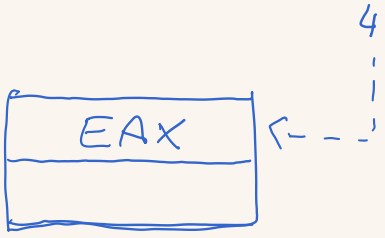
SI USANO I REGISTRI PERCHE' PIU' VELOCI E PICCOLI

SE L'ALU OGNI VOLTA CHE DEVE SVOLGERE UN'OPERAZIONE DOVESSE ACCEDERE ALLA MEMORIA IL PROCEDIMENTO SAREBBE TROPPO LENTO

METODO INDIRZZAMENTO

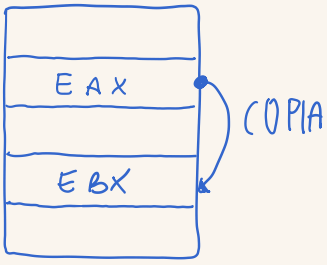
IMMEDIATO

(MOVL $4, %EAX)



A REGISTRO

MOVL %EAX, %EBX:



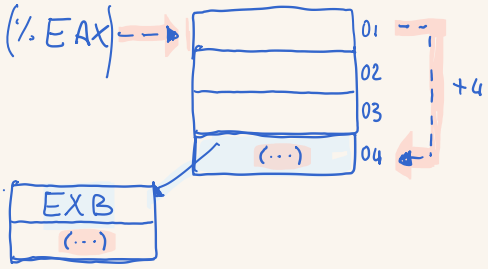
A ASSOLUTO

(MOVL %EAX, DATO)

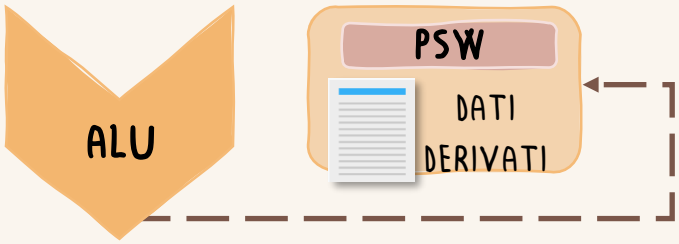


INDIRETTO A REGISTRO

(MOVL $4(%EAX), %EBX)



CPU DIDATTICA



MEMORIA



Fasi nel dettaglio:

FETCH:
- Copia il PC nel MAR
- Manda il comando READ
- Copia il MDR nel IR
- Incrementa il PC

DECODE:
- Identifica OPCODE
- Se OPCODE richiede operandi, identifica gli Operandi

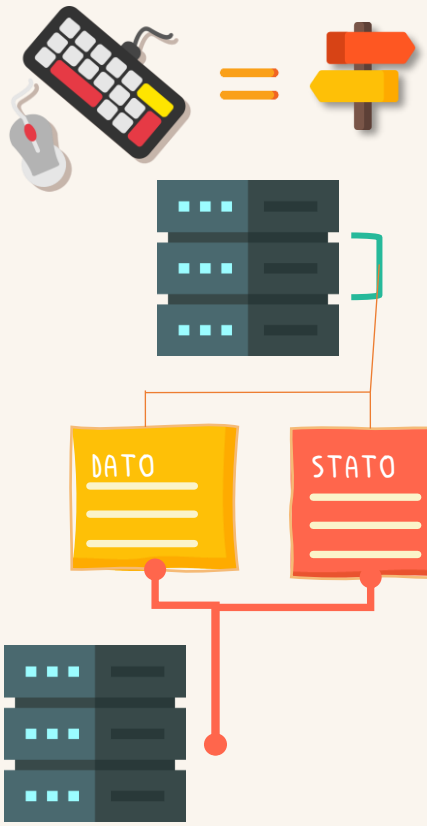
EVALUATE:
- Per le istruzioni che richiedono lettura o scrittura della memoria, calcola l'indirizzo da leggere o scrivere

FETCH OP:
- Leggi dalla memoria gli operandi richiesti

EXECUTE:
- Esegui l'istruzione richiesta

STORE:
- Scrivi, se richiesto, il risultato dell'istruzione in un eventuale registro DST.

DISPOSITIVI INPUT/ OUTPUT



IL CALCOLATORE VEDE I DISPOSITIVI I/O COME INDIRIZZI DELLA MEMORIA. QUESTA TECNICA SI CHIAMA "MECCANISMO DI MEMORY/MAPPED"

VIENE RISERVATA UNA PARTE DELLA MEMORIA DEL CALCOLATORE PER GESTIRE I DISPOSITIVI DI I/O

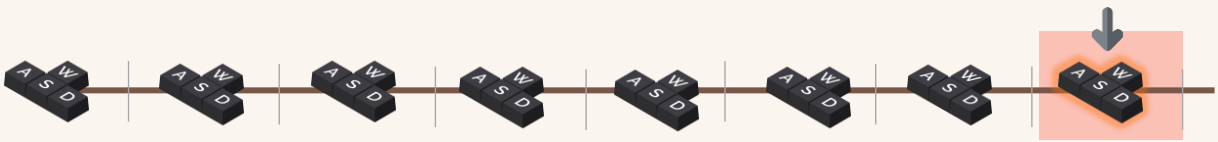
GLI INDIRIZZI ALL'INTERNO DI QUESTA AREA DI MEMORIA NON VENGONO GESTITI DALLA MEMORIA PRINCIPALE DEL CALCOLATORE, MA DA REGISTRI SPECIFICI CHIAMATI "REGISTRO DATO" E "REGISTRO STATO" DEL DISPOSITIVO DI I/O

I REGISTRI DATO E STATO HANNO UN INDIRIZZO ALL'INTERNO DELLA MEMORIA, MA NON SONO GESTITI DIRETTAMENTE DALLA MEMORIA STESSA

POLLING

```
TEST:
    MOVL INDSTATOKEY, %EAX
    CMPL $0, %EAX
    JZ TEST
    MOVL INDDATOKEY, %EBX
```

Questo script assembly può essere eseguito solo dal sistema operativo l'indirizzo di stato (INDSTATOKEY) contiene una serie di bit che forniscono informazioni sullo stato del pulsante Se tutti i bit all'interno dell'indirizzo di stato sono a 0, ciò indica che nessun pulsante è stato premuto,



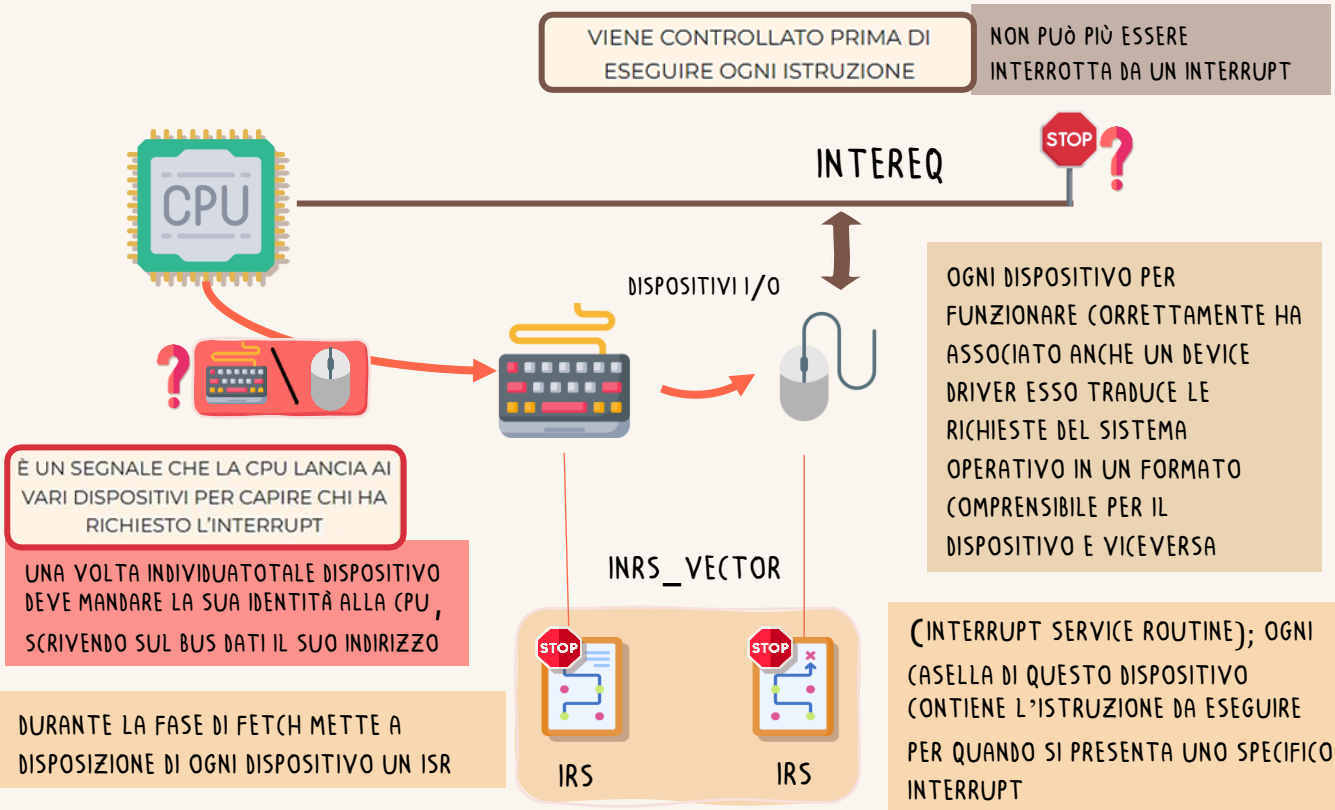
Tuttavia questo script è eseguito con una velocità enormemente superiore a quella umana, Significa che il programma che utilizza il polling controlla lo stato del pulsante in modo molto rapido e continuo. Tuttavia, questa velocità elevata può portare a uno spreco di tempo e risorse

Il polling è una tecnica utilizzata per controllare il valore di uno stato o di un dispositivo in modo continuo.

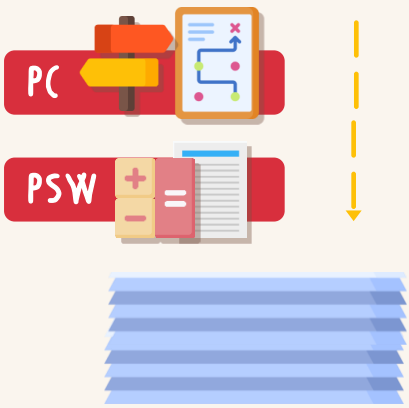
Per ovviare al problema della tecnica del Polling, si è ideato un **sistema di Interrupt.**

INTERRUPT

In questa tecnica asincrona è il dispositivo I/O che segnala alla CPU quando il tasto è premuto o quando il dato è pronto. Solo quando arriva alla CPU questo interrupt lo script TEST precedente viene eseguito.



OGNI VOLTA CHE ARRIVA UN INTERRUPT, PUÒ ESSERE RISCHIOSO.
PER RISOLVERE: A OGNI CHIAMATA DI DISPOSITIVO I/O VIENE CARICATO IL PSW SULLO STACK E IN SEGUITO IL PC. ALLA FINE VENGONO SCARICATI. QUESTO SERVE PER PREVENIRE CHE UN INTERRUPT ARRIVATO ALL'IMPROVVISO DANNEGGI L'ESECUZUONE DI UN PROGRAMMA IN CORSO.
LE INFORMAZIONI VENGONO APPUNTO SALVATE SULLO STACK E POSSONO ESSERE RILETTE NON APPENA IL DISPOSITIVO DI I/O È STATO SERVITO.





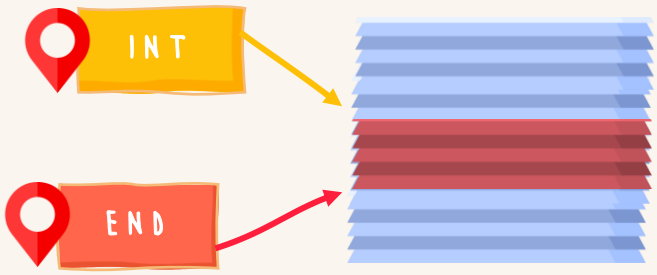
ESISTONO TUTTAVIA COMANDI CHE NON POSSONO ESSERE INTERROTTI: TALI SONO RACCHIUSI TRA CLI & STI.

↔ TIME SHARING

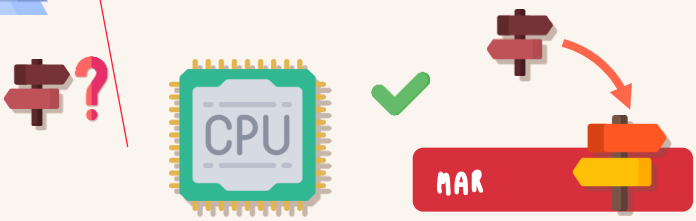


Gli interrupt sembrano superare la credenza che la cpu possa svolgere una sola operazione alla volta. Questo è possibile grazie al time sharing un programma che permette di eseguire più di un processo alla volta, viene svolto il CONTEXT SWITH(esegue un processo per un millisecondo e poi cambia).

PILA

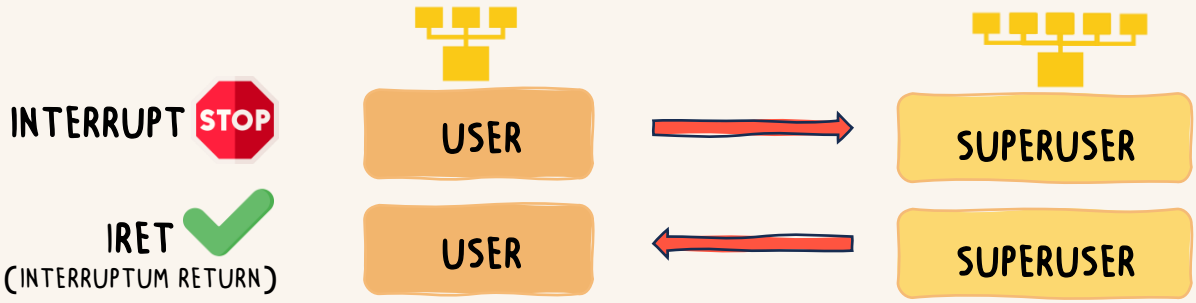


Nella memoria di un processo si hanno due registri, INIT e END. Il primo è un registro che contiene l'indirizzo di inizio dello spazio di memoria del processo, il registro END invece indica la fine del suddetto spazio di memoria



Ad ogni indirizzo che il processo chiede di accedere, la CPU controlla che questo indirizzo stia tra il valore indicato da INIT e da END. Se questo indirizzo è in questo intervallo, allora lo spedisco nel MAR, nel caso contrario la CPU chiama il sistema operativo, tramite una TRAP, per uccidere il processo.

SCHEMA DEGLI STADI E DEI PROCESSI



TIME

CON IL COMANDO TIME OTTENGO I VALORI:



PER QUANTO TEMPO IL PROCESSORE È STATO IN MODALITÀ UTENTE

PER QUANTO TEMPO IL PROCESSORE È STATO IN MODALITÀ SUPERUSER



-TEMPO REALE

RENICE

COMANDO CHE PERMETTE DI CAMBIARE LA PRIORITÀ AD UN PROCESSO IN MODALITÀ USER POSSO SOLO DIMINUIRLA



PREHAPION

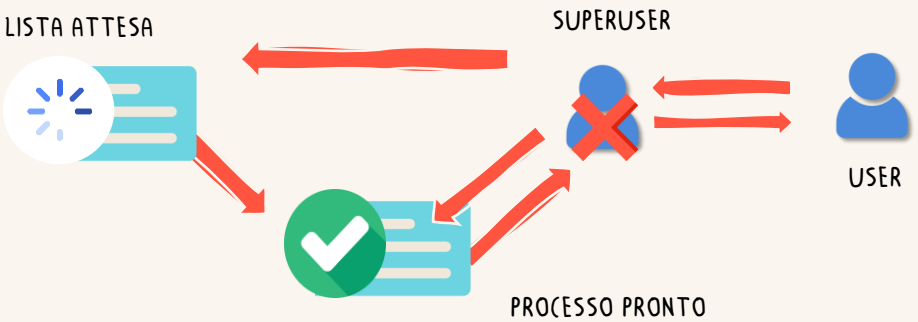
INTERROMPE L'ESECUZIONE DI UN PROCESSO ANCHE SE NECESSITEREBBE DI ESSERE ANCORA ESEGUITO. VIENE MESSO NELLA LISTA DEI PROCESSI PRONTI



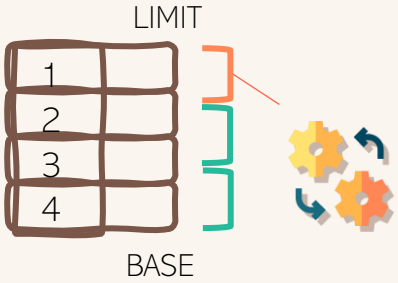
SUPER VISOR CALL

se ad un processo serve interagire con l'esterno (I/O), esso viene messo in attesa dall'interrupt, quando viene messo appunto in lista di attesa si ricorda anche
il tipo di interrupt che quel processo stà attendendo. La cpu nel frattempo può fare altre cose. Quando finalmente arriva quell'interrupt il processo passa dalla lista di attesa alla lista dei processi pronti.



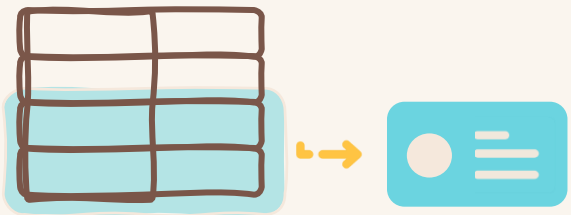


PROCESSI



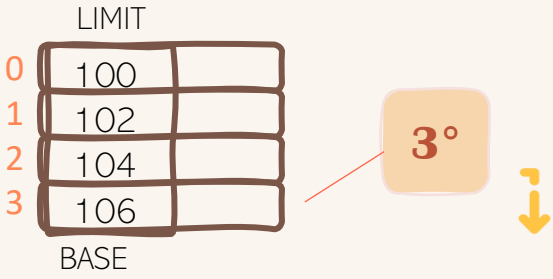
Ogni processo ha una zona di memoria dedicata a lui mentre è in esecuzione. Tale zona di memoria è limitata inferiormente dall'indirizzo BASE e superiormente dall'indirizzo LIMIT. Se il processo prova ad uscire dal suo spazio di memoria dedicato, il microproessore avverte l'OS che in genere uccide il programma

RILOCAZIONE STATICA



DECIDE UNA ZONA DI MEMORIA DA DEDICARE AL PROCESSO

RILOCAZIONE DINAMICA



BASE+ INDIRIZZO LOGICO*DIMENSIONE

100+3*2 = 106

MEMORIE

Le memorie si dividono in due grandi categorie:
-RANDOM: vi si può accedere in qualsiasi momento
-SEQUENZIALI: serve un dispositivo specifico per accedervi (dischi fissi, cd , floppy)

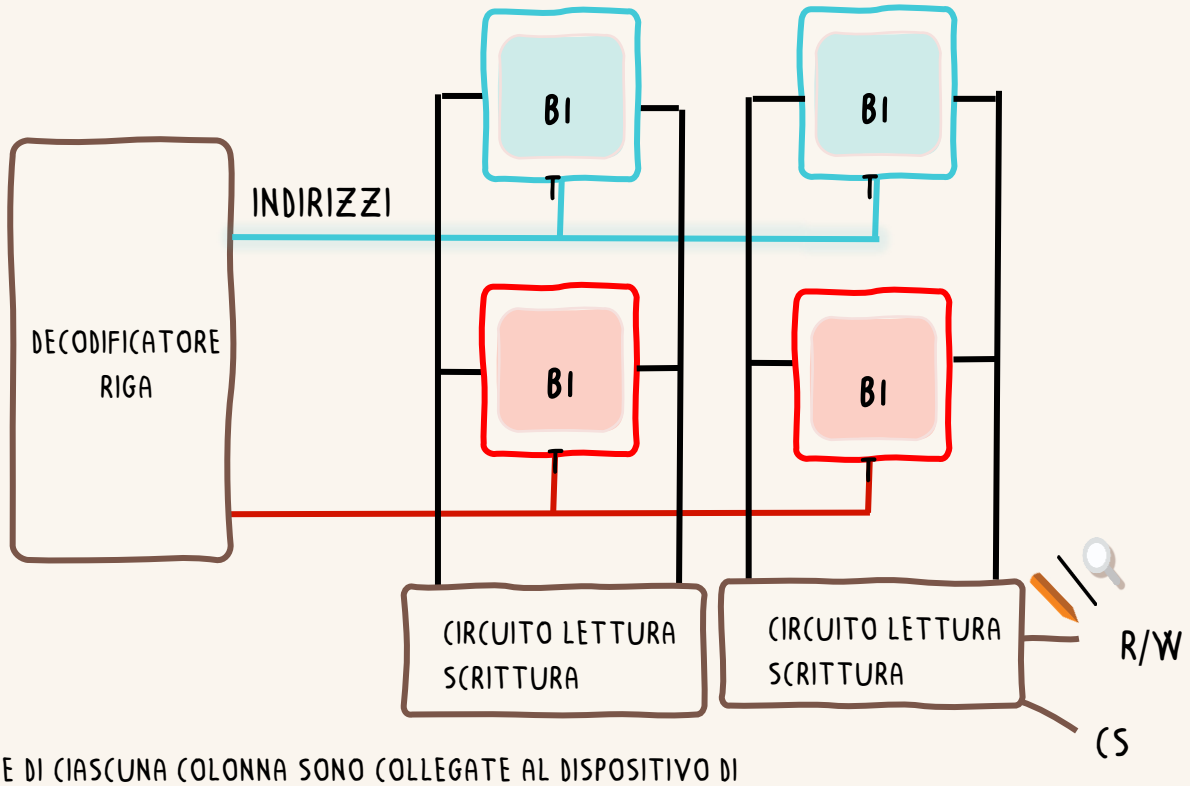
RAM

LA MEMORIA RAM SI DIVIDE IN:

SRAM (statiche)	**DRAM** (dinamiche)
Mantiene indefinitamente il valore finché è presente l'alimentazione al computer.	Perdono nel tempo l'informazione memorizzata
Volume e costo maggiori per bit.	Volume e costo minore per bit.
Consumo elettrico per bit maggiore.	Consumo elettrico per bit ridotto.
Maggiore velocità.	Minore velocità.

BANCHI DI MEMORIA

LA CELLA ELEMENTARE DI MEMORIA HA CAPACITÀ DI UN BIT E DI SOLITO LE CELLE ELEMENTARI SONO DISPOSTE COME UNA MATRICE BIDIMENSIONALE. CIASCUNA RIGA DELLA MATRICE CONTIENE UNA PAROLA DI MEMORIA E LE CELLE DA UN BIT CHE LA COSTITUISCONO SONO TUTTE COLLEGATE IN FILA MEDIANTE UNA LINEA APPOSITA COMANDATA DAL DECODIFICATORE DI INDIRIZZO

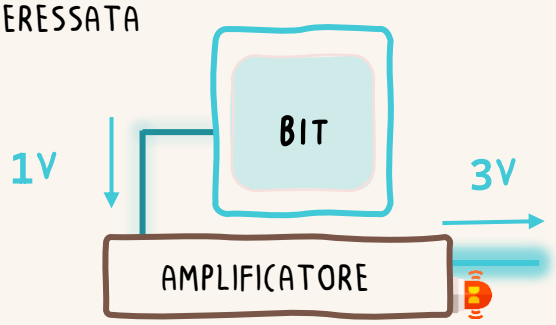


LE CELLE DI CIASCUNA COLONNA SONO COLLEGATE AL DISPOSITIVO DI LETTURA/SCRITTURA TRAMITE DUE LINEE DI BIT

DENTRO IL CIRCUITO DI LETTURA/SCRITTURA SI HA CIÒ CHE VIENE CHIAMATO AMPLIFICATORE DI SEGNALE ELETTRICO (SIGNAL AMPLIFIER), IL QUALE CONSENTE DI LEGGERE UN SEGNALE CHE SAREBBE PIUTTOSTO DEBOLE

LETTURA

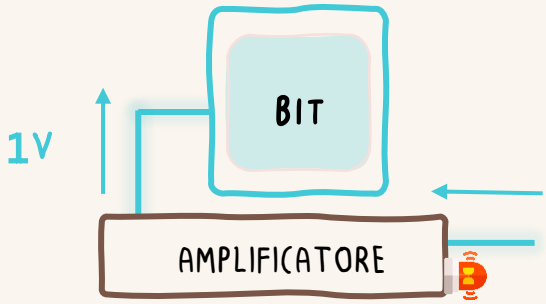
IL CIRCUITO RILEVA E AMPLIFICA IL VALORE ELETTRICO CONTENUTO NELLA CELLA INTERESSATA



E LO MANDA AL MORSETTO CORRISPONDENTE DI DATO

LETTURA

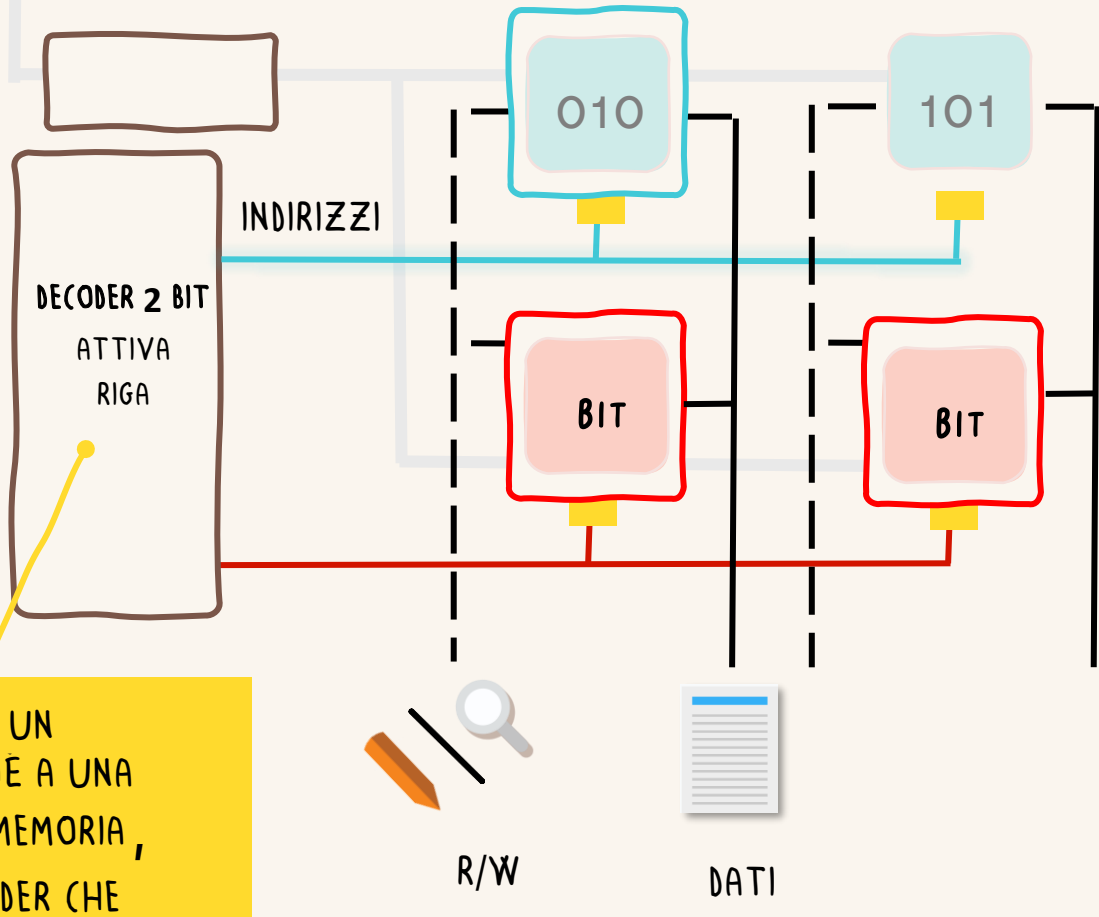
AVVIENE IL PROCESSO INVERSO, SI RILEVA LA TENSIONE PRESENTE SUL MORSETTO E LA SI VA AD APPLICARE ALLA CELLA DI MEMORIA CORRETTA



1) Si deve realizzare un banco di memoria con indirizzi a 20 bit e parole di 16 bit. S
dispone di componenti base di memoria discreti ciascuno in grado di immagazzinar
256K righe di 8 bit. Disegnare la struttura di tale modulo, identificando chiaramen
la corrispondenza tra i bit di indirizzo/dato esterni al banco ed i segnali interni.

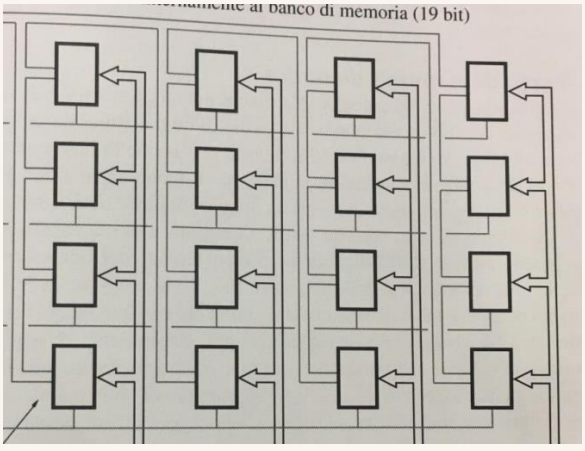
- Descrivere sommariamente le 3 condizioni necessarie affinchè la pipeline vista
lezione sia caratterizzata da CPI pari a 1 in un determinato (sotto)programm
Assumere che siano disponibili tutte le ottimizzazioni spiegate e che vi sia cach
distinta per istruzioni e dati.

I PIMI DUE BIT INDICANO LA RIGA



PER ACCEDERE A UN OTTETTO, CIOÈ A UNA PAROLA DELLA MEMORIA, SERVE UN DECODER CHE ATTIVA UNA SOLA LINEA DI RIGA DETTA WORD LINE

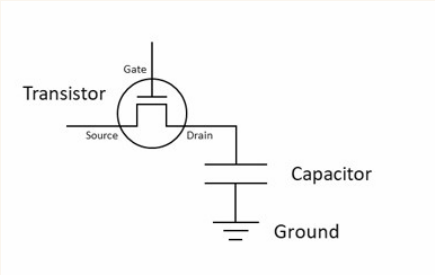
QUA ABBIAMO PER COLONNA 4 COMPONENTI DA 1 BYTE CIASCUNO CHE RAPPRESETANO UNA PAROLA DA 32 BIT



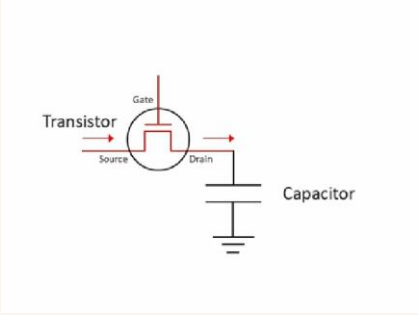
QUATTRO COLONNE AFFIANCATE COSTRUISCONO PERTANTO LA MATRICE DESIDERATA 2MX32 COMPLETA

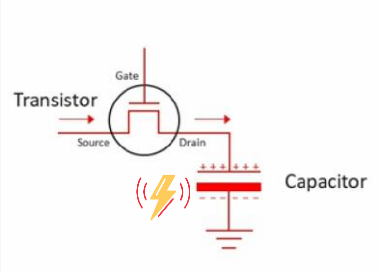
MEMORIA DINAMICA ASINCRONA

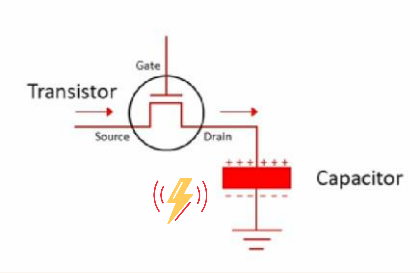
È UNA "MEMORIA DINAMICA". CHIAMATA COSI IN QUANTO NON GARANTISCE LA MEMORIZZAZZIONE PER UN TEMPO MOLTO LUNGO ANCHE QUANDO È CORRETTAMENTE ALIMENTATA. QUANDO UNA MEMORIA DINAMICA È AD ACCESSO CASUALE PRENDE IL NOME DI DRAM



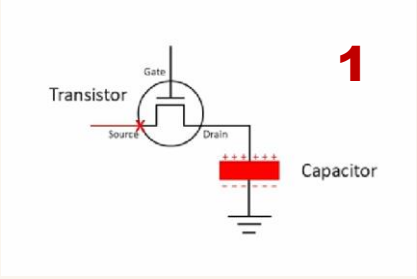
QUANDO IL GATE E' APERTO FA PASSARE IL SEGNALE CHE CARICA IL CONDESATORE





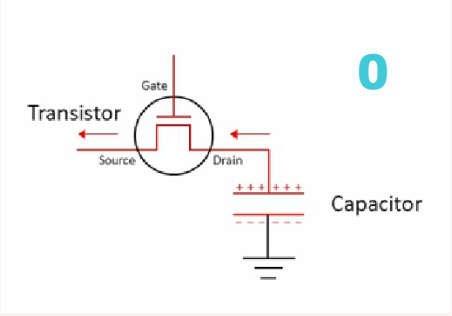


MENTRE QUANDO IL GATE E' CHIUSO NON PASSA CORRENTE E IL CONDESATORE RIMANE CARICO



QUESTO STATO RAPPRESENTA **1**

QUANDO NON PASSA PIU' LA CORRENTE DA SOURCE IL CONDESATORE RILASCIA LA CARICA E SI SCARICA

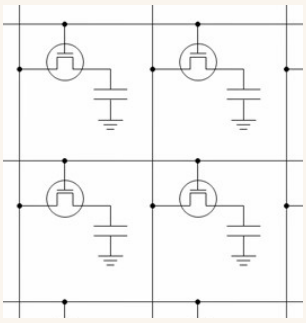


QUESTO STATO RAPPRESENTA **0**

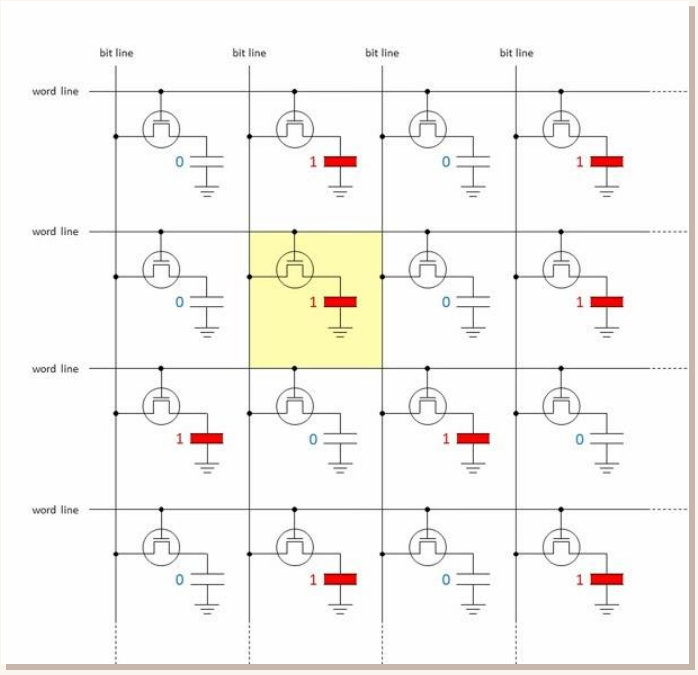
COME PUOI NOTARE LA LETTURA DI UNA CELLA DI MEMORIA E' UN PROCESSO **DISTRUTTIVO**

DRAM - Dynamic Random Access Memory

CELLA DRAM

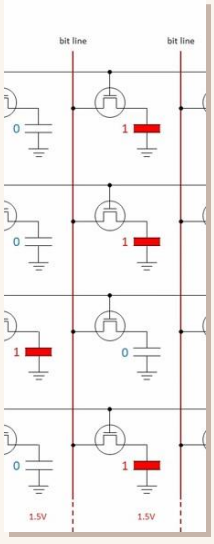


DENTRO A UNA CELLA DRAM VI E' LE CELLE DI MEMORIA SONO DISPOSTE A MATRICE

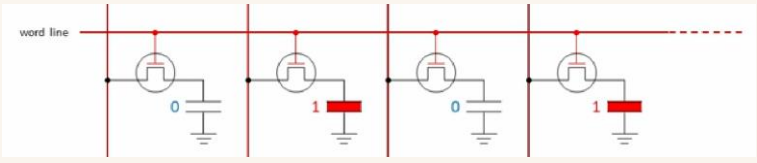


IN QUESO ESEMPIO ALCUNI CONDESATORI SONO CARICHI E METTIAMO CASA CHE VOGLIAMO LEGGERE IL CONTENUTO DI QUESTA CELLA

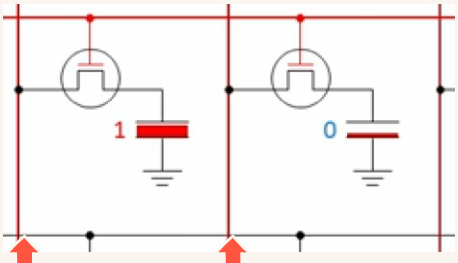
OGNI BIT LINE VIENE PRERICARICATA BREVEMENTE A META' DEL VOLTAGGIO CHE SI USA NORMALMENTE



SI AUMENTA IL VOLTAGGIO NELLA WORD LINE CORRISPETTIVA



SE UN CONDESATORE ERA CARICO PARTE DEELLA CARICA FINISCE NELLA BIT LINE , MENTRE SE ERA SCARICO PARTE DELLA CARICA FINISCE IN ESSO



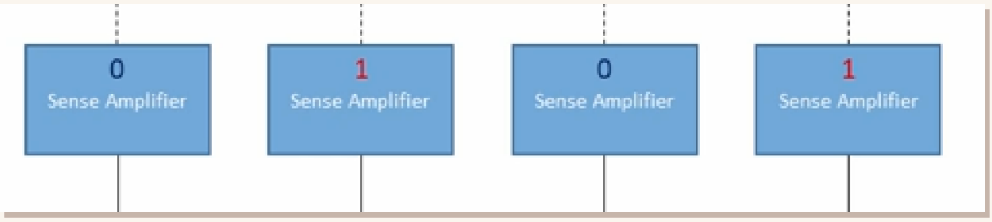
VOLTAGGIO E' AUMENTATO

VOLTAGGIO E' DIMINUITO

QUESTA PICCOLA DIFFENZA DI VOLTAGGIO VERRA' INTERCETTATA DAI AMPLIFICATORI

DRAM - Dynamic Random Access Memory

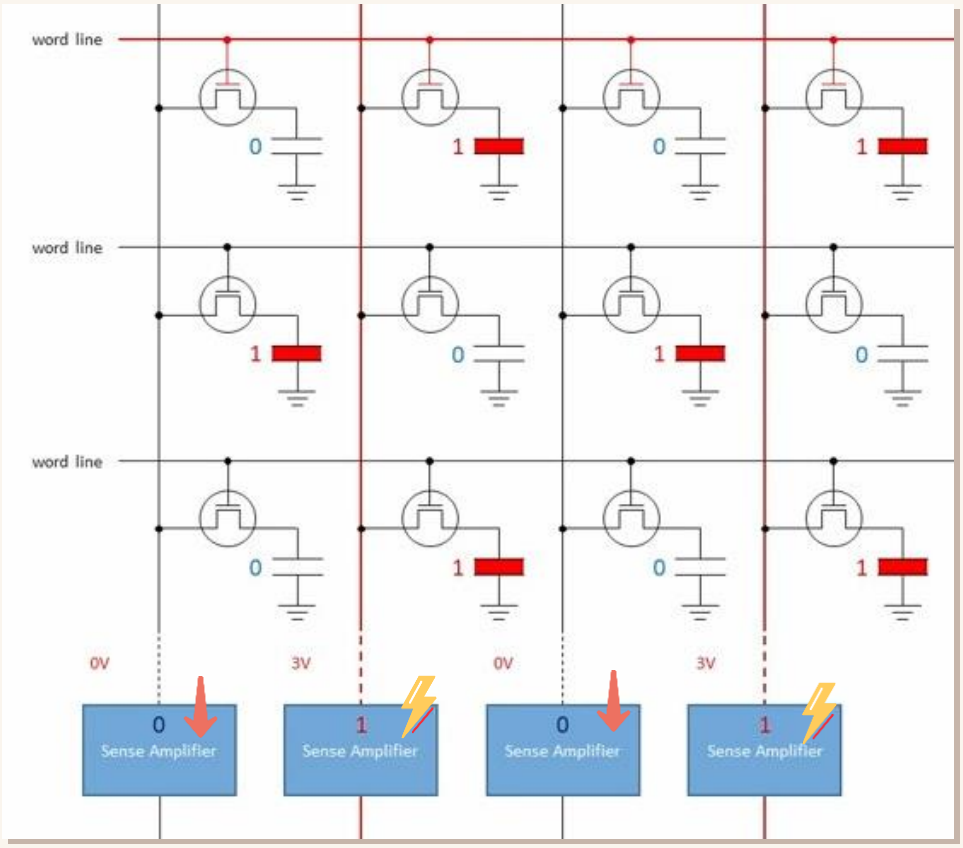
GLI AMPLIFICATORI SONO COSTITUITI DA FLIP FLOP LATCH CHE TEMPORANEMANETE CONTENGONO IL VALORE



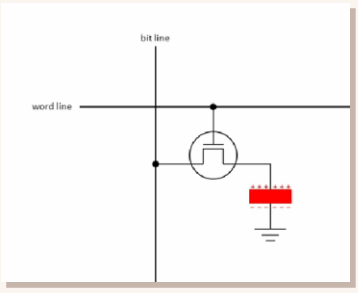
COME ABBIAMO VISTO IL PROCESSO DI LETTURA DI UNRA RIGA E' DISTRUTTIVO

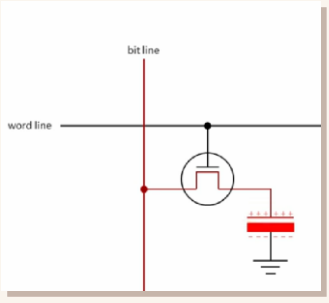
QUDINI SIGNIFICA CHE OGNI PROCESSO DI LETTURA E SEUGITO DA UNO DI RISCRITTURA

QUESTO E' UN COMPITO DEI AMPLIFICATORI CHE IN BASE AL VALORE SALVATO L IMETTONO NEI BIT LINE A PIENO VOLTAGGIO OPERATIVO O FATTO SCENDERE RISTAURANDO I VALORI PRECEDENTI



IL GATE NON E' PERFETTO E ANCHE SE NON E' ATTIVATO UN PO DI CARICA ESCE DAL CONDESATORE





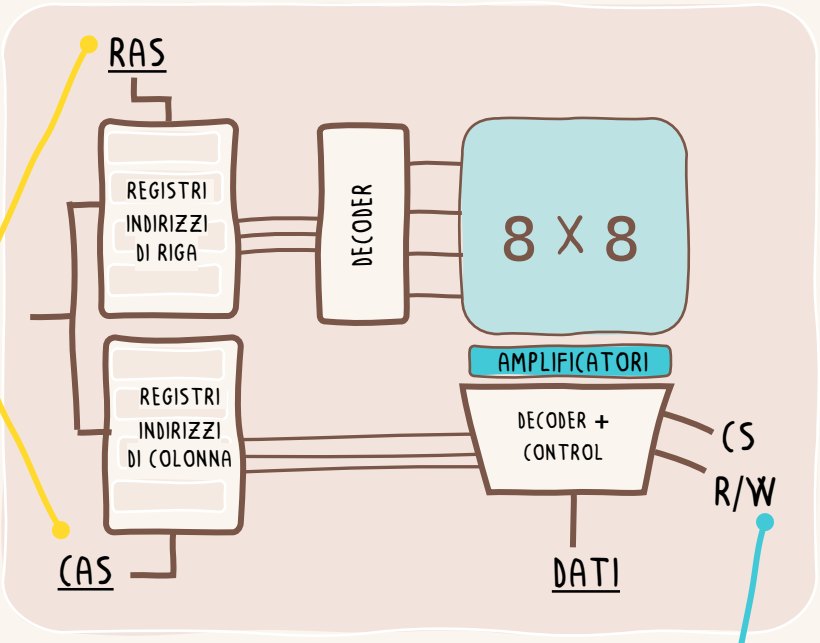
QUINDI PERIODICAMENTE LE CELLE ANDRANNO RICARICATE OVVERO LETTE E RISCRITTE ANCHE SE NON E' RICHIESTO

DRAM - Dynamic Random Access Memory

PER INDENTIFICARE UN LOCAZIONE NELLA MEMORIA ABBIAMO BISONGO DUE DECODER

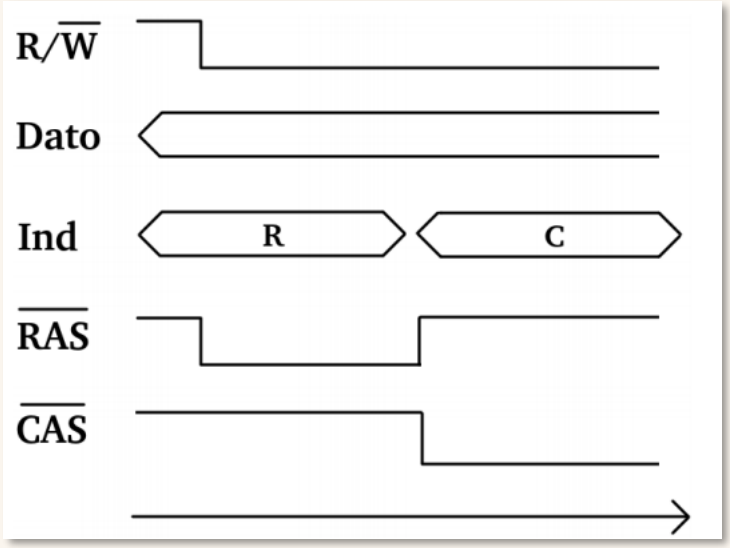
PER RAPPRESENTARE 8 INDIRIZZI CI BASTANO 3 COLLEGAMENTI ($2^3=8$)

RAS (Row Address Store) E CAS (Column Address Store) NDICANO L'INIZIO DELLA SEQUENZA DI ACCESSO AI DATI E FORNISCONO IL TIMING PER SELEZIONARE LA RIGA E LA COLONNA CORRISPONDENTI ALLE CELLE DI MEMORIA DESIDERATE PER LA LETTURA O LA SCRITTURA DEI DATI



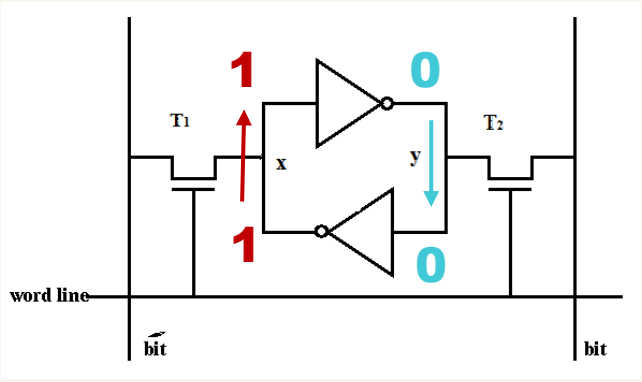
ESSENDO CHE LEGGIAMO UN SOLO BIT PER VOLTA CI BASTA SOLO UN ESTERNO COLLEGAMENTO PER INICARE LA LETTURA O SCRITTURA

PER LE DRAM SI UTILIZZA IL MECCANISMO DELLO STROBING: SI FORNISCE PRIMA CIRCA UNA METÀ DELL'INDIRIZZO E POI L'ALTRA METÀ

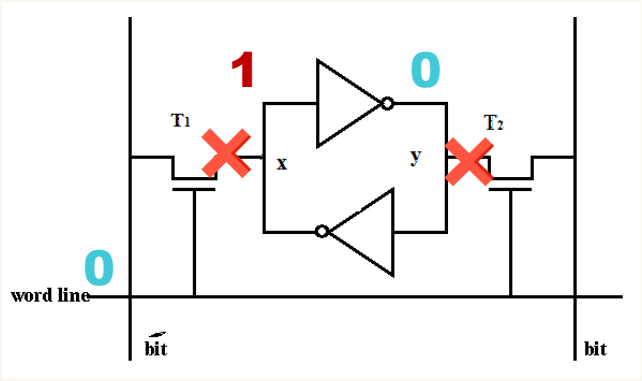


SDRAM

È UNA TECNOLOGIA DI MEMORIA CHE PERMETTE DI MEMORIZZARE IL CONTENUTO FINTANTO CHE È ALIMENTATA DA CORRENTE ELETTRICA



LA FIGURA MOSTRA UN ESEMPIO DI SRAM: IN SOSTANZA SI TRATTA DI DUE NEGATORI RETROAZIONATI LE CUI USCITE X E Y SONO COLLEGATE ALLE 2 LINEE DI BIT TRAMITE I TRANSISTOR PASSANTI T1 E T2, ESSI FUNZIONANO COME DEGLI INTERRUTTORI E SI POSSONO APRIRE O CHIUDERE PILOTATI DALLA LINEA DI PAROLA

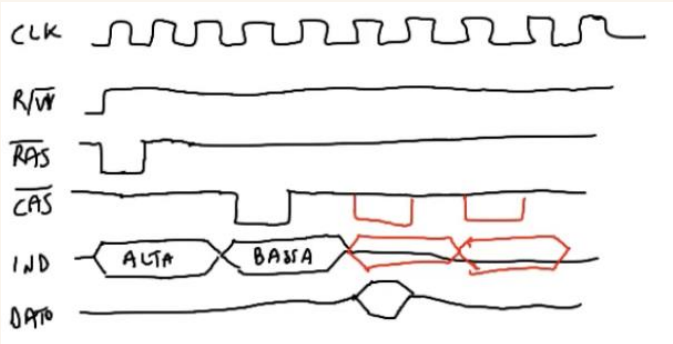


QUANDO LA LINEA DI PAROLA SI TROVA A 0 I DUE TRANSISTOR SONO APERTI DUNQUE LA COPPIA DI NEGATORI È ISOLATA E MANTIENE LO STATO

MEMORIA DINAMICA SINCRONA

LA CARATTERISTICA DISTINTIVA DI UNA SDRAM È L'UTILIZZO DI UN SEGNALE DI CLOCK CHE RENDE POSSIBILE IMPLEMENTARE UN CONTROLLORE E INCORPORARE UN SISTEMA DI REFRESH AUTOMATICO COSICCHÉ LA NATURA DINAMICA DI QUESTA MEMORIA SIA QUASI IMPERCETTIBILE.
UNA MEMORIA SDRAM PUÒ RICEVERE UN DATO PER OGNI CICLO DI CLOCK.

QUANDO HO ACCESSI CONSECUTIVI, PER ESEMPIO DA PARTE DEL DMA,
INFATTI IMPOSTO UNA SOLA VOLTA SEGUITO DA UNA SERIE DI SEGNALI COLONNA CONSECUTIVI. QUESTO RIDUCE LA LATENZA, OVVERO IL TEMPO RICHIESTO PER UN ACCESSO
CASUALE,



ARCHITETTURE PARALLELE

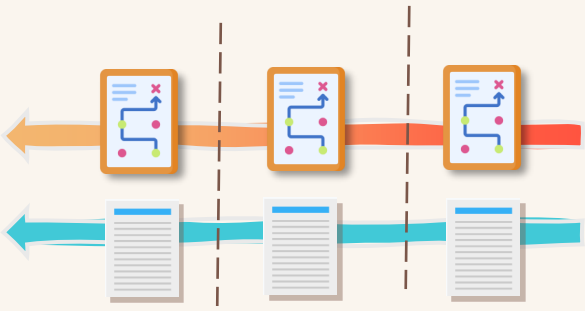
ISTRUZIONI/DATI	SINGLE	MULTI
SINGLE	SISD	MISD
MULTI	SIMD	MIMD

RILOCAZIONE DINAMICA

SISD

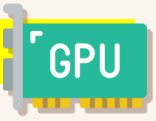
Von Neumann

PROCESSA UN SINGOLO INSIEME DI DATI E UN UNICO FLUSSO DI ISTRUZIONI.

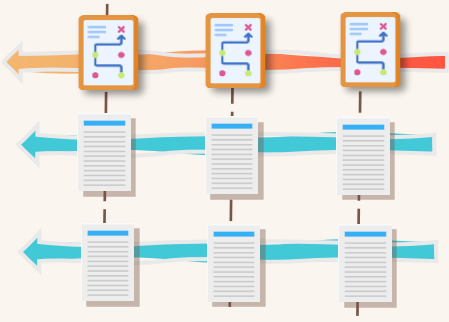


SIMD

GPU

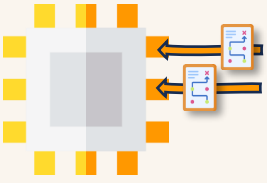


Single Instruction, Multiple Data

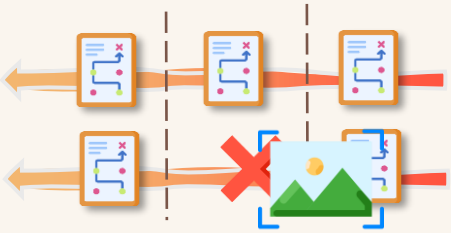


PROCESSORE IN GRADO DI GESTIRE **SINGOLE ISTRUZIONI** MA SU **DATI MULTIPLI**

L' BIETTIVO DI UN' ARCHITETTURA SIMD E ACCELERARE L ELABORAZIONE PARALLELA DI GRANDI QUANTITÀ DI DATI CONSENTE A UN PROCESSORE DI ESEGUIRE LA STESSA OPERAZIONE SU PIÙ DATI CONTEMPORANEAMENTE.



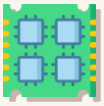
OGNI ALGORITMO HA UNA SERIE DI ISTRUZIONI CHE POSSONO ESSERE E FATTE IN PARALLELO E UNA SERIE CHE NON POSSONO ESSERE FATTE IN PARALLELO, QUESTE ULTIME SONO LA MAGGIORANZA.



MISD

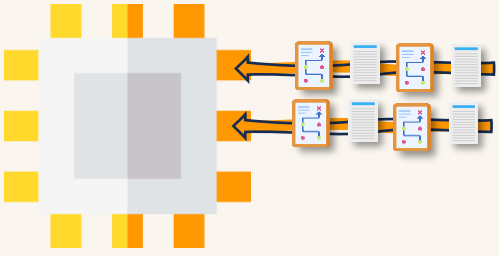


MIMD

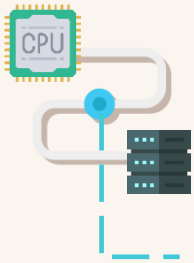


multicore

OGNI PROCESSING UNIT LAVORA CON UN INSIEME DI ISTRUZIONI E DI DATI DIVERSI.

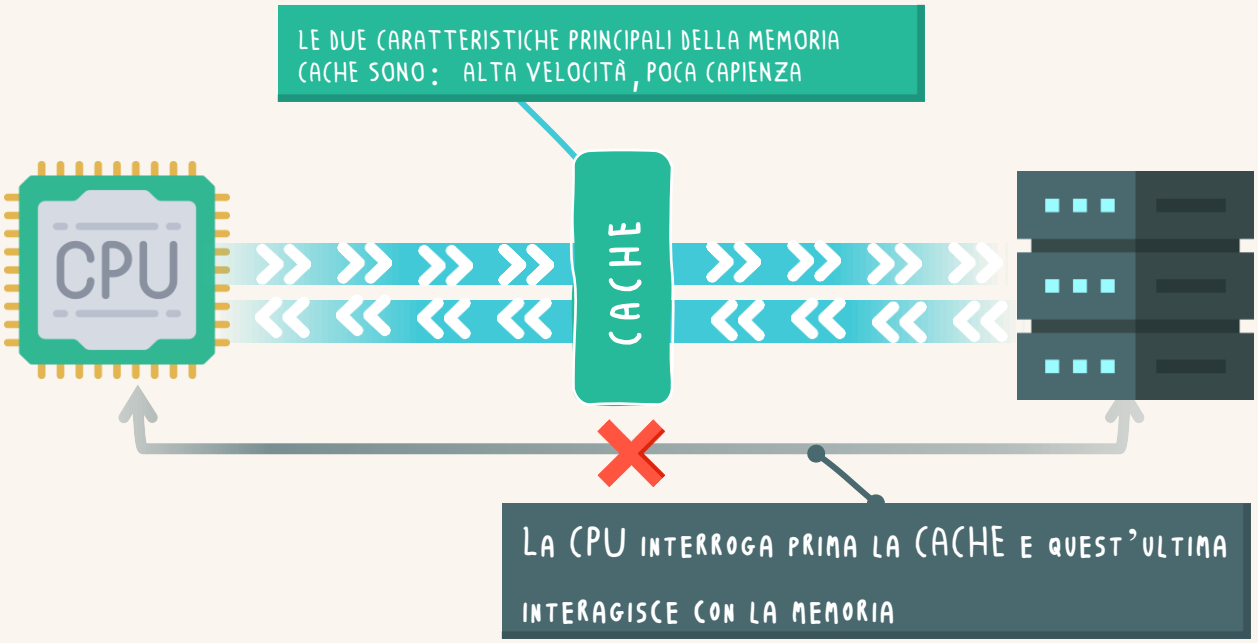


CACHE



LA CPU È IN GRADO DI COMUNICARE CON LA MEMORIA ATTRAVERSO I BUS, TUTTAVIA LA MEMORIA LA CPU DA SOLA NON PUÒ GESTIRE, È' LENTA. OGNI VOLTA CHE SI ATTUA UNA LOAD O STORE AVVIENE UN RITARDO SIGNIFICATIVO

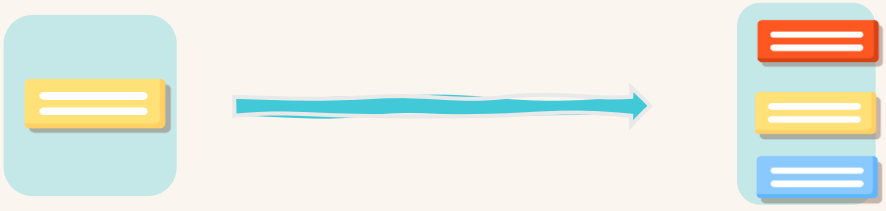
LA CACHE HA L'OBBIETTIVO DI PORSI A METÀ STRADA TRA CPU E MEMORIA, FORNENDO MENO MEMORIA DI QUELLA CENTRALE, MA PIÙ VELOCE



PRINCIPIO DI LOCALITÀ NELLA CACHE



SECONDO QUESTO PRINCIPIO, È' MOLTO PROBABILE CHE ENTRO UN BREVE PERIODO DI TEMPO SI AVRÀ BISOGNO DI UNA PAROLA IN MEMORIA CHE SIA VICINA A QUELLA A CUI SI ACCEDE ATTUALMENTE. PER SFRUTTARE QUESTO PRINCIPIO, QUANDO SI RICHIEDE UNA SPECIFICA POSIZIONE DI MEMORIA ALL'INTERNO DI UN BLOCCO, CONVIENE CARICARE L'INTERO BLOCCO NELLA CACHE ANZICHÉ SOLO LA SINGOLA PAROLA



CACHE

LA CPU LEGGE O SCRIVE LA CACHE

- **CACHE Hit**: la parola è in CACHE
- **CACHE Miss:** la parola è in memoria.

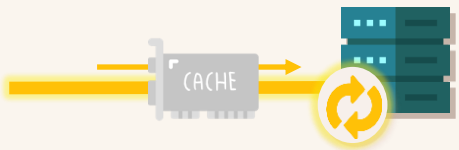


	Hit	Miss
Read	Prelievo da CACHE	Read-Back Load-Through
Write	Write-Back Write-Through	Write-Back Write-Through

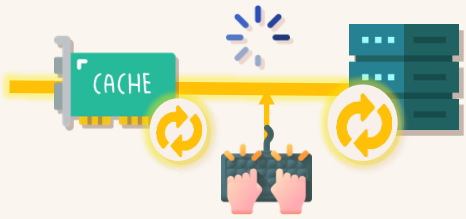
HIT



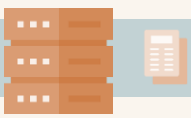
- **Write-Through:** aggiorna la memoria.



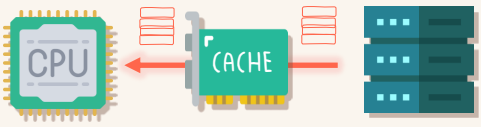
- **Write-Back**: non aggiorna subito la memoria, ma imposta a 1 il dirty bit. Successivamente si aggiorna la memoria e il dirty bit torna a 0 quando avviene un trasferimento tra I/O e memoria che coinvolge una posizione avente dirty bit a 1.



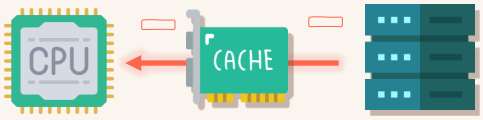
MISS



- **Read-Back**: trasferisce tutto il blocco da memoria a CACHE, poi la parola da CACHE a CPU.

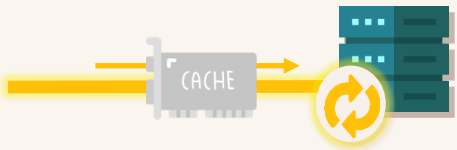


- **Load-Through**: trasferisce prima la parola da memoria a CACHE, poi da CACHE a CPU mentre si lascia completare il trasferimento del blocco da memoria a CACHE.

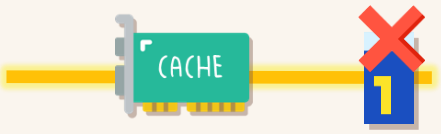


CACHE

HIT



• **Write-Through**: aggiorna la memoria direttamente, senza coinvolgere la CACHE.



• **Write-Back**: implica che si carichi in CACHE il blocco, si aggiorna in CACHE la parola e il dirty bit va a 1

RICAPITOLANDO

CACHE HIT — IL DATO E' PRESENTE IN CACHE

• **LETTURA**: carico in cpu la parola richiesta dalla posizione individuata



ABCD

• **SCRITTURA IMMEDIATA** : aggiorno sia CACHE che memoria subito (semplice dal punto di vista dell'architettura ma deve fare tante scritture quante istruzioni)

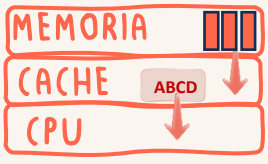


• **SCRITTURA DIFFERITA** : : aggiorno la CACHE suBITo ma la memoria no; setto un BIT (modify BIT) a 1 per ricordare ad accessi futuri che la memoria non è aggiornata. Il modify BIT è associato alla posizione del blocco. (questo processo ha una scrittura costosa computazionalmente m scrive meno volte.)

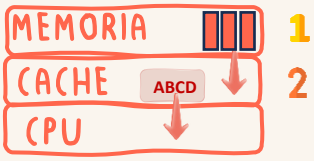


CACHE MISS — IL DATO CHE SAREBBE SERVITO NON SI TROVA IN CACHE

• **LETTURA IMMEDIATA:** nello stesso istante in cui carico il blocco dalla memoria alla CACHE, carico la parola da CACHE a cpu.

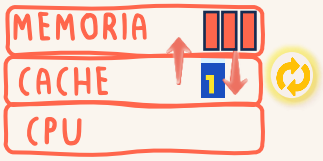


• **LETTURA DIFFERITA:** carico prima il blocco da memoria a CACHE e in seguito carico la parola desiderata in cpu.



• **SCRITTURA IMMEDIATA:** scrivo direttamente in memoria senza passare dalla CACHE.

• **SCRITTURA DIFFERITA**: carico il blocco interessato da memoria a CACHE, aggiorno tale blocco in CACHE e setto il modify BIT a 1. Successivamente lo riporto in memoria.



ESERCIZI MEMORIA CACHE

DIMENSIONE INDIRIZZO

ABCD
ABCD
ABCD

per fare ciò, bisogna semplicemente prendere il numero di parole nella RAM e scomporlo in potenza di 2. Si otterrà perciò un numero 2^n , il cui "n" sarà il numero corrispondente alla dimensione dell'indirizzo

Sapere quanto valgono, in potenze di 2, 1KB, 1MB, 1GB ecc.. può agevolare l'operazione:

1KB = 1024 bit = 2^{10} 1MB = 1024 kb = 2^{20}

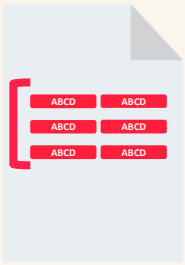
1GB = 1024 mb = 2^{30} 1TB = 1024 gb = 2^{40}

Se quindi il numero di parole nella Ram è 4M parole:

$4 = 2^2$

$1MB = 2^{20}$

$2^{20} * 2^2 = 2^{22}$ 22 bit (Dimensione dell'indirizzo)



DIMENSIONE DEL CAMPO PAROLA

per fare ciò, bisogna considerare il numero di parole per blocco e trasformarlo in potenza di 2. Si ottiene perciò il numero 2^n , il cui "n" è il numero corrispondente alla dimensione del campo PAROLA.

DIMENSIONE DEL CAMPO BLOCCO

è necessario ottenere la dimensione della Cache (a volte indicata con "# Cache", altre con "dim. Cache"); per far ciò, occorre dividere il numero di parole nella Cache per il numero di parole per blocco.